EXPERIMENT – 14

Title

Write a Program to Create a Voting Application using ReactJS

Aim

To develop a **Voting Application** using ReactJS that allows users to vote for candidates and display live vote counts dynamically.

Prerequisites

- Basic knowledge of JavaScript (ES6)
- Understanding of ReactJS components
- Knowledge of useState Hook
- Node.js installed

Software Requirements

- OS: Windows / Linux / macOS
- Node.js (LTS)
- ReactJS
- VS Code
- Browser (Chrome)

Theory

React is used to create interactive UIs using components and state management.

Concepts Used:

- Functional Components
- `useState` Hook
- Event Handling (`onClick`)
- Conditional Rendering

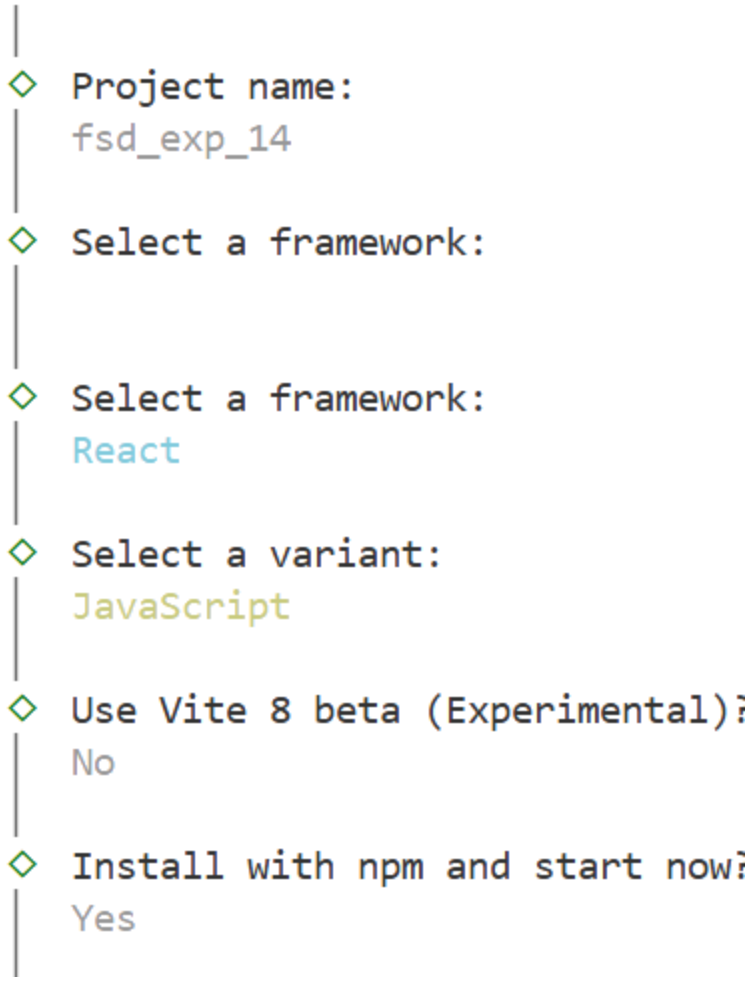
In this experiment:

- Users can vote for candidates
- Votes are updated dynamically
- Winner can be displayed

Project Setup

Step 1: Create React App

```shell
npm create vite@latest
cd voting-app
npm start
```



Source Code

Replace `src/App.js` with:

```javascript
import React, { useState } from "react";
import "./App.css";

function App() {

  const [votes, setVotes] = useState({
```

```
    CandidateA: 0,
    CandidateB: 0,
    CandidateC: 0
  });

  const voteFor = (candidate) => {
    setVotes({
      ...votes,
      [candidate]: votes[candidate] + 1
    });
  };

  const getWinner = () => {
    const maxVotes = Math.max(...Object.values(votes));
    const winners = Object.keys(votes).filter(
      candidate => votes[candidate] === maxVotes
    );
    return winners.join(", ");
  };

  return (
    <div className="container">
      <h2>React Voting Application</h2>

      <div className="candidate">
        <h3>Candidate A</h3>
        <p>Votes: {votes.CandidateA}</p>
                                <button    onClick={()    =>
voteFor("CandidateA")}>Vote</button>
      </div>

      <div className="candidate">
        <h3>Candidate B</h3>
```

```
        <p>Votes: {votes.CandidateB}</p>
                                <button    onClick={()    =>
voteFor("CandidateB")}>Vote</button>
      </div>

      <div className="candidate">
        <h3>Candidate C</h3>
        <p>Votes: {votes.CandidateC}</p>
                                <button    onClick={()    =>
voteFor("CandidateC")}>Vote</button>
      </div>

      <hr />

      <h3>Current Winner: {getWinner()}</h3>
    </div>
  );
}

export default App;
```

App.css

Replace `src/App.css` with:

```css
.container {
  text-align: center;
  margin-top: 40px;
}
```

```
.candidate {
  margin: 20px;
  padding: 15px;
  border: 1px solid black;
  display: inline-block;
  width: 200px;
}

button {
  padding: 8px 12px;
  cursor: pointer;
}
```

Execution Steps

Run:

```Shell
npm start
```

Open:

```None
http://localhost:3000
```

Functionality Demonstrated

Feature	Description
Vote Button	Increases vote count
State Update	Dynamic re-render
Winner Calculation	Highest vote count
Real-time Display	Live vote tracking

Sample Output

Initial:

```
None
Candidate A: 0
Candidate B: 0
Candidate C: 0
```

After Voting:

```
None
Candidate A: 3
Candidate B: 2
Candidate C: 1

Current Winner: Candidate A
```

Expected Result

The Voting Application successfully updates vote counts dynamically and displays the current winner using ReactJS state management.

Result

Thus, a Voting Application was successfully developed using ReactJS with dynamic state updates.

Viva Voce Questions

1. What is useState in React?
2. Why do we use spread operator?
3. What is re-rendering?
4. How does React update UI automatically?
5. What is conditional rendering?
6. Can this be connected to a backend?

Conclusion

This experiment demonstrates ReactJS fundamentals such as state management, event handling, and dynamic UI updates through a Voting Application.